

Read Well Assessment App

Technical Requirements Document · Prepared September 22, 2026

1. Technical Overview

This Technical Requirements Document (TRD) translates the PRD for the Read Well Assessment App into a concrete technical approach: a web application with a relational backend that stores students, assessments, and results, and a reporting layer that auto-generates a PDF report per student and a rolled-up PDF report per school. The system is built as a standard three-tier web app (frontend, backend/API, database) with a dedicated report-generation service, deployed as a single web application to start (no native mobile app in this release, per the PRD).

Design principles:

- Favor a managed backend (auth, database, storage) to minimize infrastructure overhead for a first release
- Keep the difficulty-area → recommendation mapping data-driven/configurable rather than hard-coded, so the program team can maintain it without a deploy
- Design the data model to extend cleanly to additional grade levels in later phases

2. Recommended Tech Stack

Layer	Recommendation	Rationale
Frontend	Next.js (React + TypeScript)	Single framework for both the student assessment UI and teacher/assessment management UI
Backend / API	Supabase (Postgres + Auto-generated APIs + Edge Functions) or Next.js/TypeScript API if notes, and storage; i.e. avoid	Fast prototyping, working on a single platform
Database	PostgreSQL (via Supabase)	Relational model fits the domain well; mature, well-understood
Auth	Supabase Auth (or equivalent) with role-based access	Needed for the role-based report access defined in the PRD
Report Generation	Server-side PDF generation triggered after scoring	Produces the exportable per-student and school-wide PDF reports
File Storage	Supabase Storage (or equivalent object storage)	Stores generated PDF reports for retrieval/export
Hosting	Vercel (frontend + API routes)	Pairs naturally with Next.js; simple deploy pipeline

Note: This stack favors speed-to-MVP using managed services. If the school/district requires on-premise hosting or has existing infrastructure standards, those constraints should be confirmed before finalizing this choice (see Open Questions).

3. Core Data Model

Entity	Key Fields	Notes
School	id, name, address	Top-level tenant for aggregate reporting
User	id, name, email, role, school_id	Role drives access control
Student	id, name, grade, school_id, teacher_id	Grade drives which assessment is assigned
Assessment	id, grade_level, version, items (config)	Content is grade-specific and configurable
AssessmentSession	id, student_id, assessment_id, status, started_at, completed_at	Supports autosave/resume
Response	id, session_id, item_id, answer, is_correct	Raw captured answers, scored per item
SkillArea	id, name (phonics, fluency, comprehension, ...)	Reference table for scoring breakdown
Result	id, session_id, skill_area_id, score, flagged_as_difficult	Per-skill area scoring output
RecommendationRule	id, skill_area_id, grade_level, recommendation_text	Configurable mapping from difficulty → recommendation
StudentReport	id, student_id, session_id, generated_at, pdf_url	Generated per-student report
SchoolReport	id, school_id, cycle_id, generated_at, pdf_url	Generated aggregate report

Relationships: School 1–N Student; Student 1–N AssessmentSession; AssessmentSession 1–N Response and 1–N Result; Result N–1 SkillArea; SkillArea 1–N RecommendationRule; StudentReport 1–1 AssessmentSession; SchoolReport aggregates across StudentReports for a School within a cycle.

4. Key API / Service Boundaries

4.1 Assessment Service

- POST /assessments/sessions — start a session for a student (server resolves the correct assessment by student grade)
- PATCH /assessments/sessions/:id — autosave responses during the session
- POST /assessments/sessions/:id/complete — mark complete and trigger scoring

4.2 Scoring Service

- Triggered on session completion
- Scores responses per skill area against the grade-level rubric
- Writes Result rows and flags difficulty areas below threshold
- Triggers report generation on completion

4.3 Recommendation Engine

- Given a set of flagged skill areas + grade level, looks up matching RecommendationRule rows
- Returns the recommendation text/content references used in the student report
- Rules are managed as data, not code, so the program team can update them

4.4 Report Generation Service

- generateStudentReport(sessionId) — renders and stores a PDF, updates StudentReport
- generateSchoolReport(schoolId, cycleId) — aggregates data for the cycle, renders and stores a PDF, updates SchoolReport
- Runs asynchronously (background job/queue) so it doesn't block the assessment-completion response

5. Report Generation Pipeline

- 1 Assessment session completes → Scoring Service scores responses and writes Result rows
- 2 Scoring Service calls the Recommendation Engine for each flagged skill area
- 3 Report Generation Service renders the per-student report (HTML/template → PDF) using student info, scores, flagged areas, recommendations
- 4 Generated PDF is stored (object storage) and linked via StudentReport.pdf_url
- 5 On a scheduled or triggered cycle close, the School Report job aggregates all StudentReport/Result data for the school and renders the school-wide PDF
- 6 Both reports use the confirmed brand palette (sage green, cream, and the confirmed accent color) in their PDF templates

Technical requirement: Report generation must be decoupled from the assessment-completion request (async/queued) so a slow render never blocks the student's or teacher's UI.

6. Authentication & Authorization

- Authentication via a managed auth provider (e.g., Supabase Auth) supporting school staff logins
- Role-based access control (RBAC) with roles: Teacher, Reading Specialist, Administrator
- Teacher: read/write access to their own students' sessions; read access to their own students' reports
- Reading Specialist: read access to reports for students assigned to them
- Administrator: read access to all students and reports within their school
- Student assessment sessions are launched in a controlled/kiosk-style context (teacher-initiated) rather than requiring students to independently authenticate
- All access rules enforced at the API/database layer, not just hidden in the UI

7. Non-Functional / Technical Requirements

Category	Requirement
Performance	Assessment UI interactions respond in under ~300ms; report generation completes within a few minutes of assessment completion
Scalability	Support concurrent assessment sessions from an entire grade/school (dozens to low hundreds of simultaneous sessions)
Reliability	Autosave assessment progress at least every response; session state recoverable after network loss or device restart
Offline Handling	Assessment client should tolerate brief connectivity drops without losing in-progress answers (local buffering + sync on reconnect)
Data Retention	Student results and reports retained per school/district data retention policy (to be confirmed)
Observability	Logging/monitoring on scoring and report-generation pipelines to catch failed report generation before a teacher sees a broken report

8. Security & Student Data Privacy

- Student data classified as sensitive education data; handled per applicable requirements (e.g., FERPA or local equivalent)
- Encryption in transit (TLS) and at rest for all student data and generated reports
- Role-based access enforced server-side, not just in the UI
- Generated report PDFs stored with access controls matching the underlying student's report permissions, not publicly accessible by guessable URL
- Audit logging of who accessed or exported a given student's report
- No student data used for purposes outside assessment/reporting without explicit school/district agreement

9. Infrastructure & Deployment

- Environments: development, staging, production, each with isolated databases
- Hosting: Frontend + API on Vercel; database/auth/storage on Supabase (or equivalent managed provider)
- CI/CD: Automated build, test, and deploy pipeline on merge to main; staging deploys for review before production
- Backups: Automated daily database backups with a defined retention window
- Background jobs: A queue or scheduled job mechanism for asynchronous report generation
- Monitoring: Error tracking and uptime monitoring on the API and report-generation pipeline

10. Open Technical Questions

- 1 Does the school/district require on-premise hosting, or is a managed cloud stack (Supabase/Vercel) acceptable?
- 2 Are there existing systems (SIS, single sign-on) this app must integrate or authenticate with, even if not in Phase 1?

- 3 What is the expected concurrent load (students per assessment window) to size infrastructure correctly?
- 4 What are the confirmed student data retention and deletion requirements?
- 5 Who owns and maintains the RecommendationRule data going forward — the app team or the reading program team?
- 6 What PDF rendering approach is preferred/approved given any existing tooling constraints?